

Name : Asiful Islam
ID : 2037820127
DEPT : Computer Science & Engineering
Course Title : Digital System Design
Course Code : CSE 2111

VHDL = (VHSIC Hardware Description Language)

VHDL ~~has many~~ is a programming language that has been designed and optimized for describing the behavior of digital circuits and systems. As such, VHDL combines features of :

- A simulation modeling language
- A Design Entry language
- A Test language
- A Netlist language
- A standard language

8. (a) what is VHDL? Briefly describe the salient features of VHDL and Verilog.

Ans: VHDL stands for Very-High-Speed Integration circuit HDL (Hardware Description Language). It is an IEEE (Institute of Electrical and Electronics Engineers) standard hardware description language that is used to describe and simulate the behavior of complex digital circuits.

The most popular examples of VHDL are Odd Parity Generator, Pulse Generator, Behavioral Model for 16 words, 8 bit RAM, etc.

Verilog is the HDL that is completely emerging and evolving in which new features are getting added continuously. VHDL is a strongly typed language and is very verbose while Verilog is a weakly typed language and has all the predefined datatypes with it.

(c) Write a VHDL code to implement XOR operation.

Ans:

VHDL XOR :

Syntax : $c \leq a \text{ xor } b;$

Purpose :

A XOR gate is also known as an exclusive OR.

Think of it as a simple OR, where if either inputs are a 1, the output is a 1. Except, when both inputs 1 is, the output is a 0. The truth table is given below:

A (in)	B (in)	c (out)
0	0	0
0	1	1
1	0	1
1	1	0

There are situations where you want this logical behaviour, and it's easy enough to implement in code. A common use is when you want to detect either a rising or a falling edge on a signal.

Here's how to use a XOR to detect either a rising or falling edge on a signal.

```
PROC_EDGE_DETECT : process (clk, reset)
begin
```

```
    if rising_edge(clk) then
```

```
        if (reset = '1') then
            input_1q <= '0';
```

```
        else
```

```
            -- delay input by 1 clock
            input_1q <= input;
```

```
            -- detect either edge
            edge_detect <= input_1q xor input;
```

```
        end if;
```

```
    end if;
```

```
end process;
```

If you need the edge-detect above to be aligned with input-19, you can move the edge-detect $\angle = \text{input-19} \text{ XOR input; line outside of the process and make it concurrent.}$

b) Explain an 8-to-1 line multiplexer module in Verilog HDL.

Ans:

An 8-to-1 multiplexer is a digital device that selects one of the eight inputs lines to the output line by using three-bit selection line. The block diagram of 8-to-1 MUX is shown in Figure 1. A 2^n -to-1 multiplexer needs n bit selection line to select one of the 2^n inputs to the output.

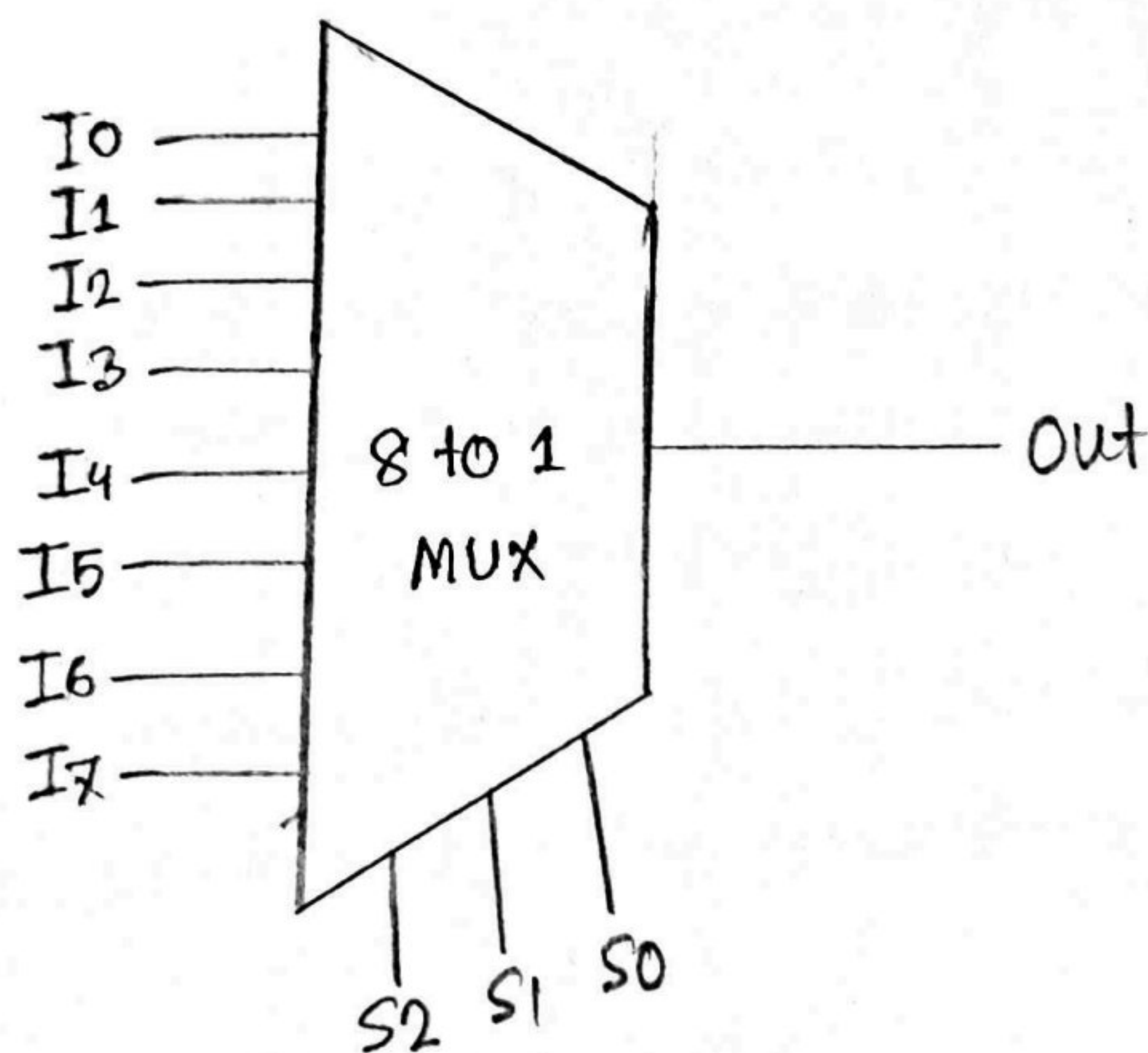


Figure 1. Block diagram of 8-to-1 multiplexer

Figure 2 Shows the truth table of the 8-to-1 multiplexer. I₀ to I₇ are the input lines, S₁-S₃ are the selection lines and O is the output line.

S ₁	S ₂	S ₃	O
0	0	0	I ₀
0	0	1	I ₁
0	1	0	I ₂
0	1	1	I ₃
1	0	0	I ₄
1	0	1	I ₅
1	1	0	I ₆
1	1	1	I ₇

Figure 2. Truth table of 8-to-1 multiplexer.

Figure 3 Shows the Verilog module of the 8-to-1 multiplexer. The 8-bit Ports In₁ to In₈ are input lines of the multiplexer. The Sel Ports is the 3-bit Selection line which is required to select between the eight input lines. 8-bit Port Out is the output line of the multiplexer.

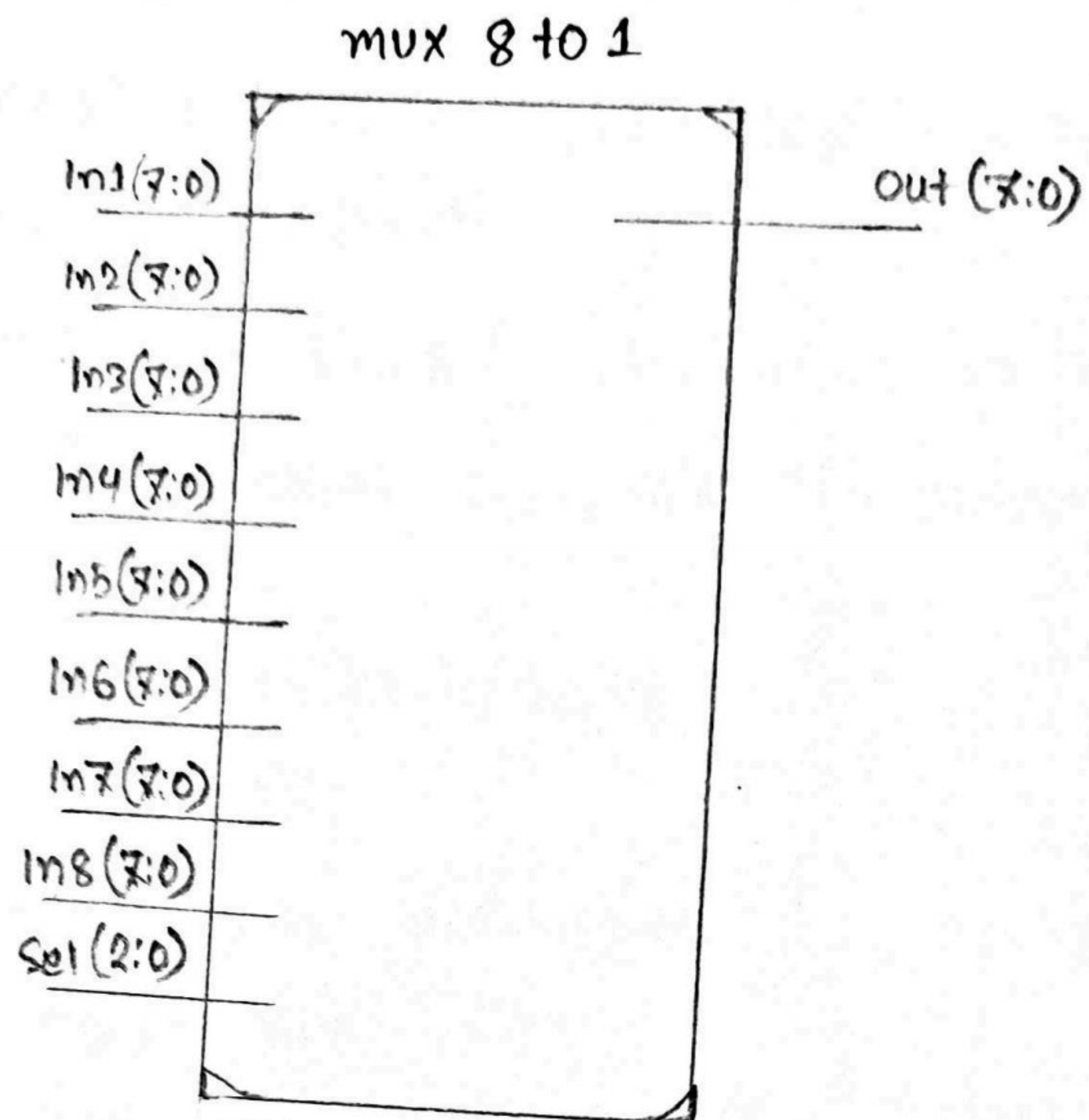


Figure 3. Verilog module of 8-to-1 multiplexer.

Declaration : (8:1 Multiplexer)

// Module 8-to-1 multiplexer. Port list is taken exactly from
// the I/O diagram.

module mux8-to-1 (out, i0, i1, i2, i3, i4, i5, i6, i7, s2, s1, s0);

// Port declarations from the I/O diagram

Output out;

input i0, i1, i2, i3, i4, i5, i6, i7;

input s2, s1, s0;

// Internal wire declarations

wire s2n, s1n, s0n;

wire y0, y1, y2, y3, y4, y5, y6, y7;

// Gate instantiations

// Create s2n, s1n and s0n signals.

not(s2n, s2);

not(s1n, s1);

not(s0n, s0);

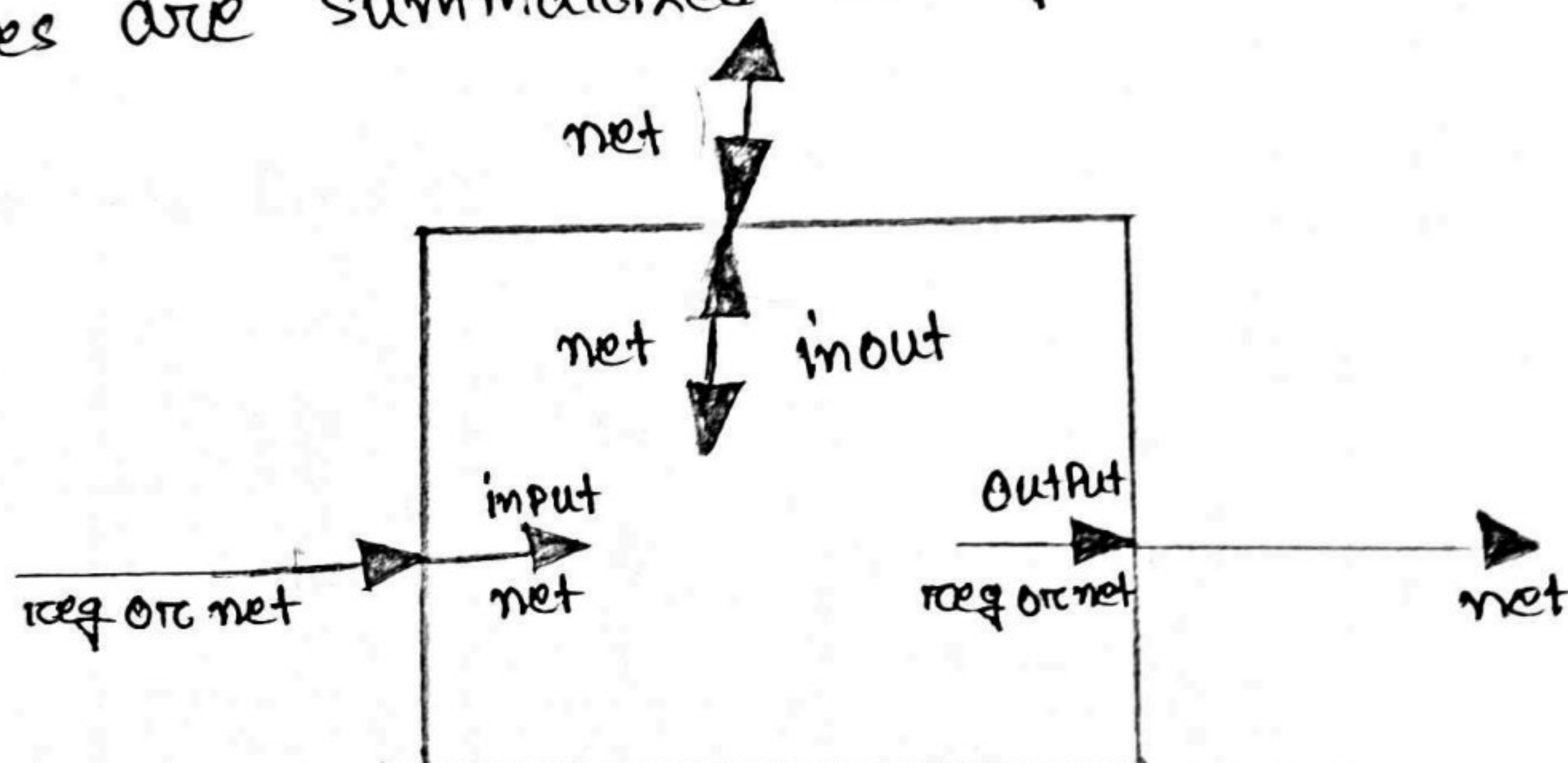
not complete

(b) Describe the "Port Connection rule" used in Verilog HDL.

Ans:

Port connection Rules :

One can visualize a Port as consisting of two units, one unit that is internal to the module and another that is external to the module. The internal and external units are connected. There are rules governing port connections when modules are instantiated within other modules. The Verilog Simulator complains if any port connection rules are violated. These rules are summarized in Figure 1 :



Inputs

Internally, input ports must always be of the type net. Externally, the inputs can be connected to a variable which is a reg or a net.

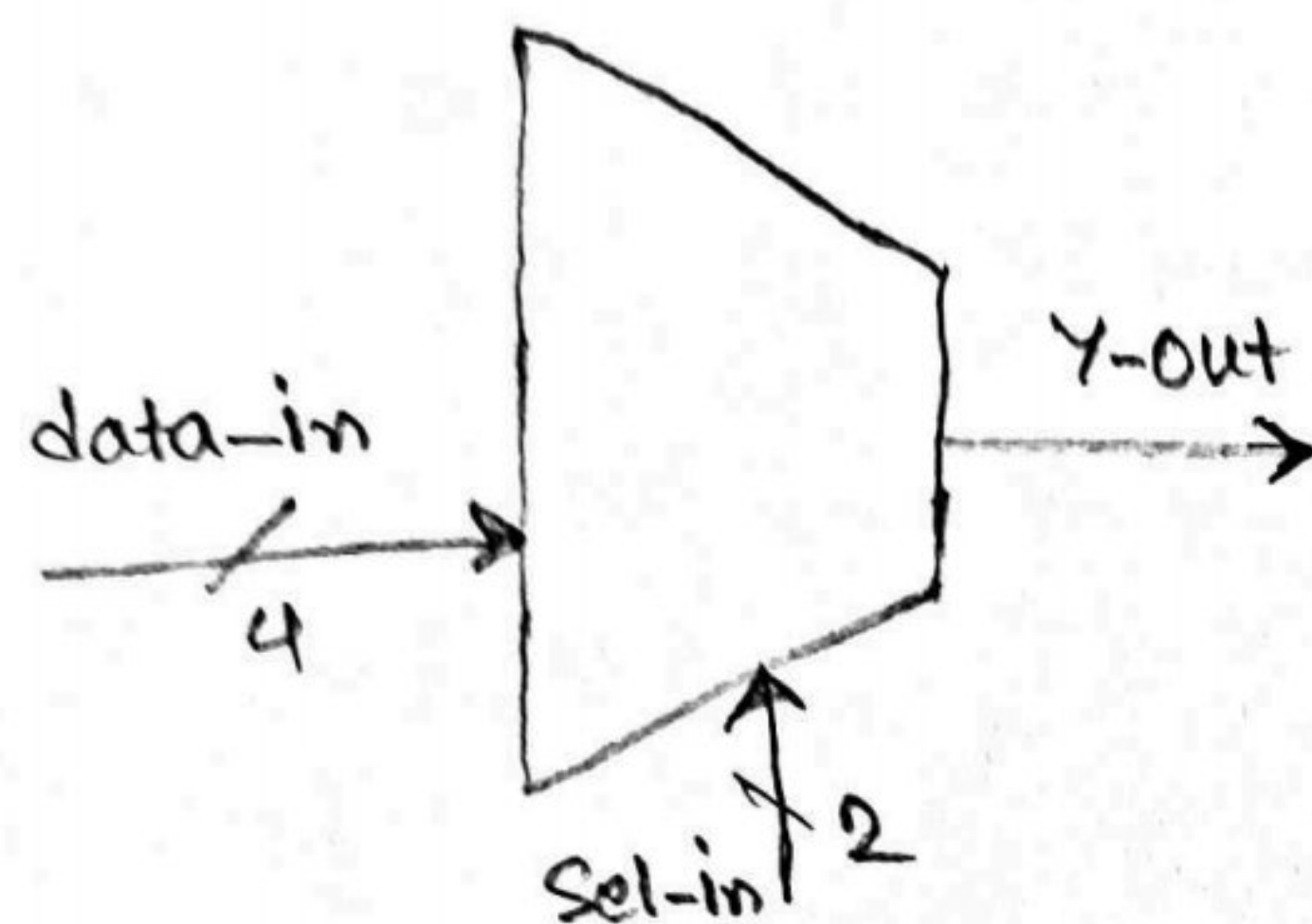
Outputs

Internally, output ports can be of the type reg or net. Externally, outputs must always be connected to a net. They cannot be connected to a reg.

(c) write a program in Verilog HDL for a 4-to-1 Multiplexer.

Ans:

Selects a data signal from the bus data-in based on select line sel-in to the output signal y-out.



Functional Table :

sel-in		y-out
[1]	[0]	
0	0	data-in [0]
0	1	data-in [1]
1	0	data-in [2]
1	1	data-in [3]

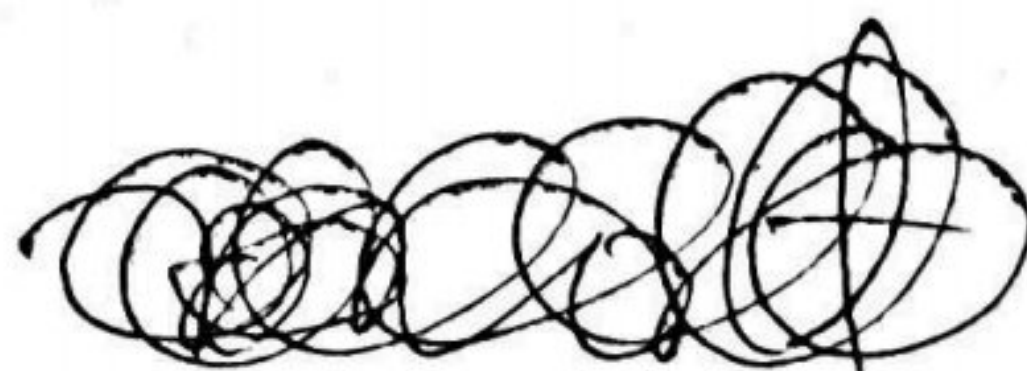
Program:

```
module mux_4_1 (data-in, sel-in, y-out);
```

```
input [3:0] data-in;
```

```
input [1:0] sel-in;
```

```
output reg y-out;
```



```
always (data-in, sel-in)
```

```
begin
```

```
case (sel-in)
```

```
2'b00 : y-out = data-in[0];
```

```
2'b01 : y-out = data-in[1];
```

```
2'b10 : y-out = data-in[2];
```

```
2'b11 : y-out = data-in[3];
```

```
default : y-out = 0;
```

```
endcase
```

```
end
```

```
endmodule
```

Q) what are the "Design Block" and "Stimulus Block" used in Verilog HDL?

Ans:

There are two main blocks for writing Verilog HDL programming. These are called the design block and stimulus block.

The design block is used to model/represent the functionality of the circuit/IC to be simulate.

Example: For AND gate simulator, it represents the functionality of AND gate (input-output).

A stimulus block, on the other hand, performs the simulation based on the preset simulation parameters (in/out variables, timing, etc.) and show the output based on the design specified in design block.

Name: Most: Aoborn Begum

ID : 1937820109

Dept: Computer Science & Engineering

Course Title: Digital System Design

Course Code: CSE 2112

7(b) 24/11/22

Sil Afroz
11/09/22

Previous Year Question - 2020
Part-B (5,7)

5.(a) Implement a full adder circuit using 3 to 8 decoder.

Ans: A decoder with an OR gate at the output can be used to implement the given boolean function. The decoder should at least have as many input lines as the number of variables in the boolean function to be implemented. The truth table of the full adder is given below:

Truth table:

A	B	C	S	C ₀	
0	0	0	0	0	m ₀
0	0	1	1	0	m ₁
0	1	0	1	0	m ₂
0	1	1	0	1	m ₃
1	0	0	1	0	m ₄
1	0	1	0	1	m ₅
1	1	0	0	1	m ₆
1	1	1	1	1	m ₇

From the truth table, we can get this equation:

Sum output, $S = \Sigma 1, 2, 4, 7$

Carry out, $C_o = \Sigma 3, 5, 6, 7$

Circuit diagram of full adder using 3 to 8 decoder given below:-

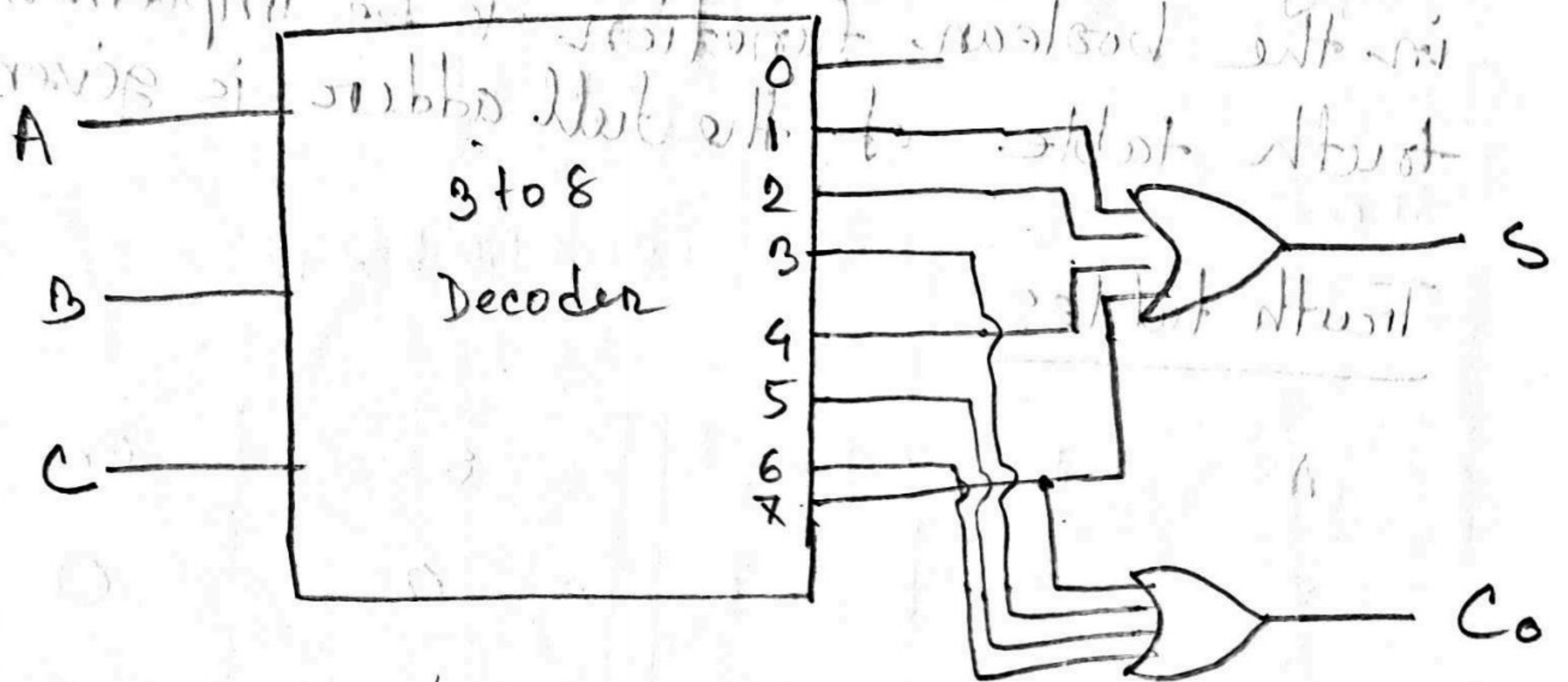


Fig: (1) Full adder using 3 to 8 decoder.

b) How decoder can be used as de-multiplexer.

Ans: Decoder is form of n to 2^n line decoder having n input lines and 2^n line output with enable line. For this decoder to work as demultiplexer, its enable pin is used as input and n input lines of decoder as select lines and 2^n output lines.

Now, let's see the block diagram of decoder can be used as a demultiplexer.

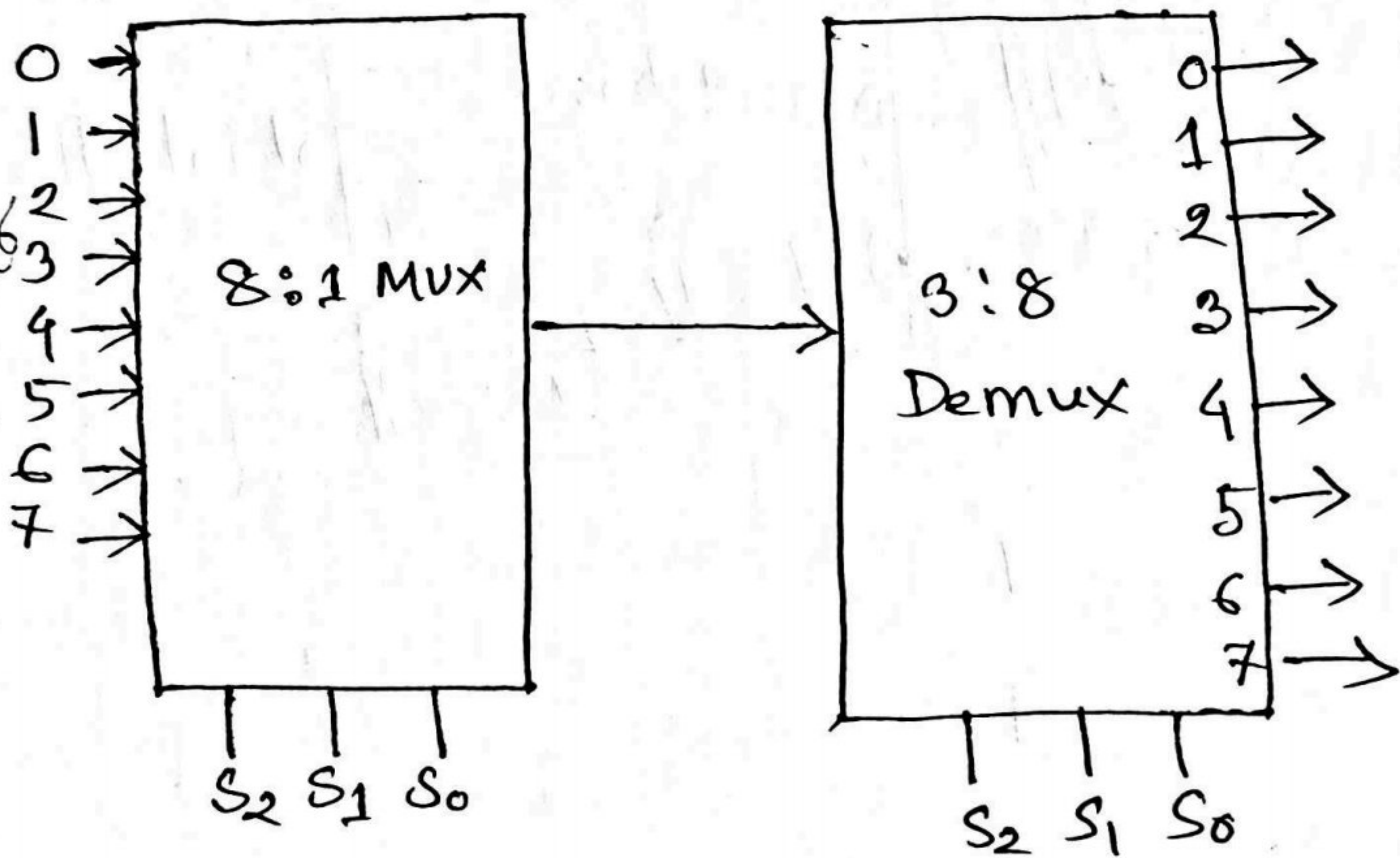
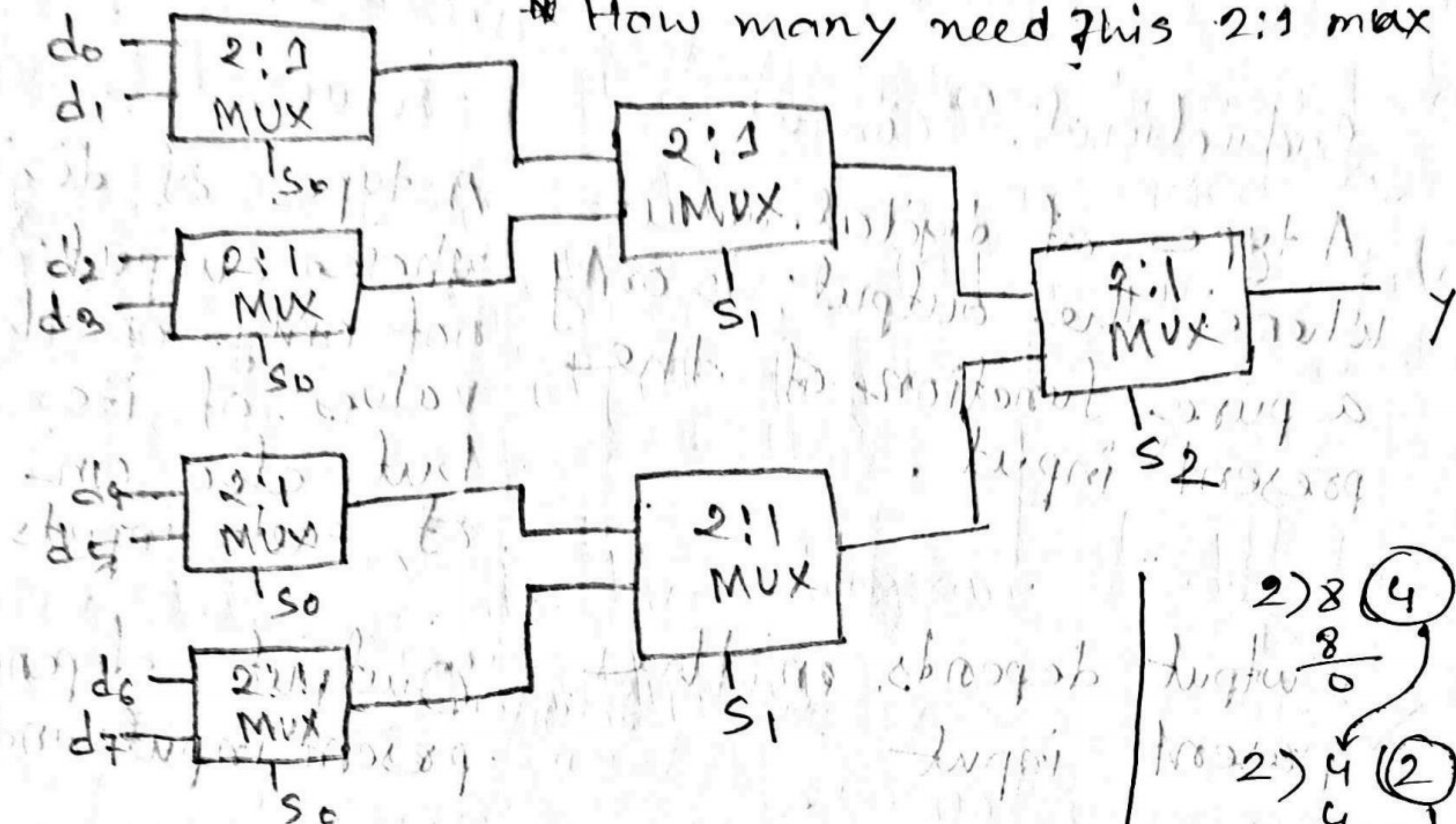


Fig: (i) Block diagram of decoder can be used as a demultiplexer.

(1) How could we implement an 8 to 1 multiplexer using 2 to 1 multiplexers. 1st calculation: $2^3 = 8$ available = 2:1 mux.
 How many need this 2:1 mux?



Truth table: 2:1 Multiplexer

x	y	f	f=x
0	0	0	f=x
0	1	0	
1	0	0	
1	1	1	f=y

$$\begin{array}{r}
 2) 8 \quad (4) \\
 \underline{8} \\
 0 \\
 2) 4 \quad (2) \\
 \underline{4} \\
 0 \\
 2) 2 \quad (1) \\
 \underline{2} \\
 0
 \end{array}$$

total 2:1 mux need = $4 + 2 + 1 = 7$.

d) Distinguish between combinational and sequential logic circuit.

Combinational circuit	Sequential circuit
A type of digital circuit where the output is only a pure function of the present input.	A type of digital circuit whose output depends on not only on the present values of its input signals but also on the sequence of past inputs
Output depends on the present input	Output depends on the present input and past outputs
There is no memory unit.	There is a memory unit to store immediate results
There is no clock	There is a clock.
Example: Half adder, full adder, MUX, De-MUX etc	Ex: flip flop and registers, counter



7.(a) What are advantages of synchronous counter over asynchronous counter?

Ans: The one advantage of synchronous counter over asynchronous counter is, it can operate on higher frequency than asynchronous counter as it does not have cumulative delay because of same clock is given to each flip flop.

It has no associated propagation delay. Operation is faster than with an asynchronous counter. It's easier to design than the Asynchronous counter. Because the count sequence is regulated by logic gates, the possibilities of an error are reduced.

7. (v) Design a 4 bit synchronous counter to over for counting the following binary sequence

Ans: $0000 \rightarrow 0101 \rightarrow 1010 \rightarrow 0110 \rightarrow 1001 \rightarrow 0011 \rightarrow 1100 \rightarrow 0000$

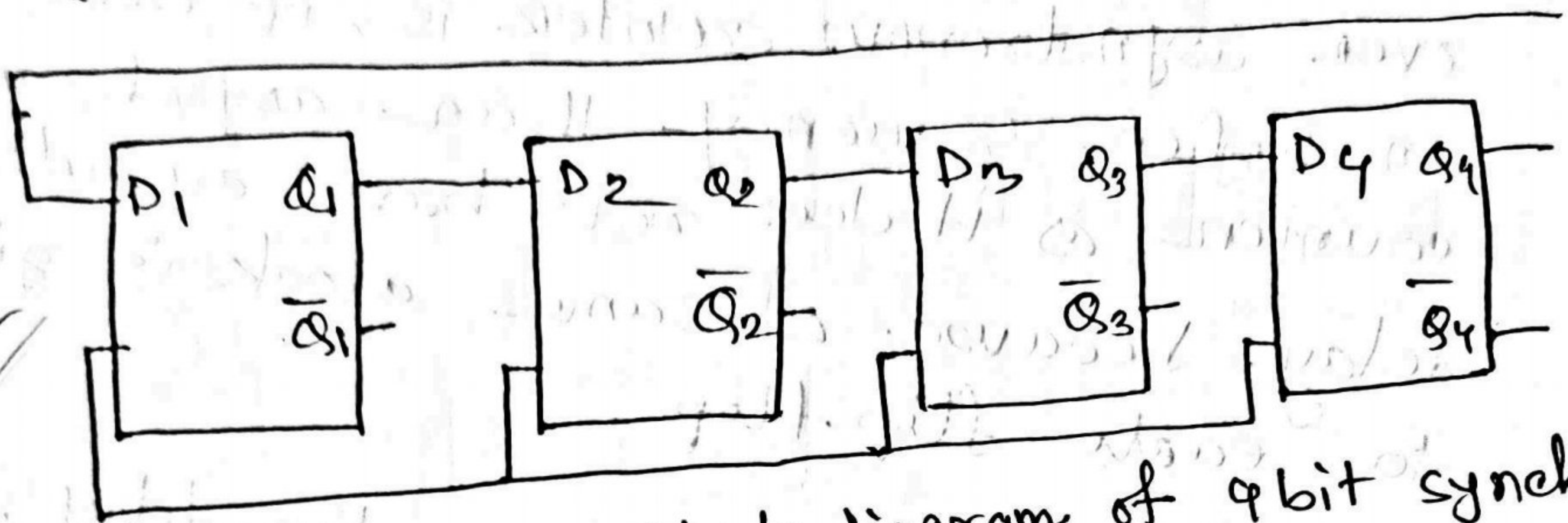


Fig: (v), Block diagram of 4 bit synchronous counter

The counter is obtained from a serial-in, serial-out shift register by providing feedback from the inverted output of the last FF to the D input of the first FF. The Q output of each edge is connected to the D input of the next stage, but the output of the last stage is connected to the D input of first stage, therefore the name twisted ring counter.

Twisted ring counter

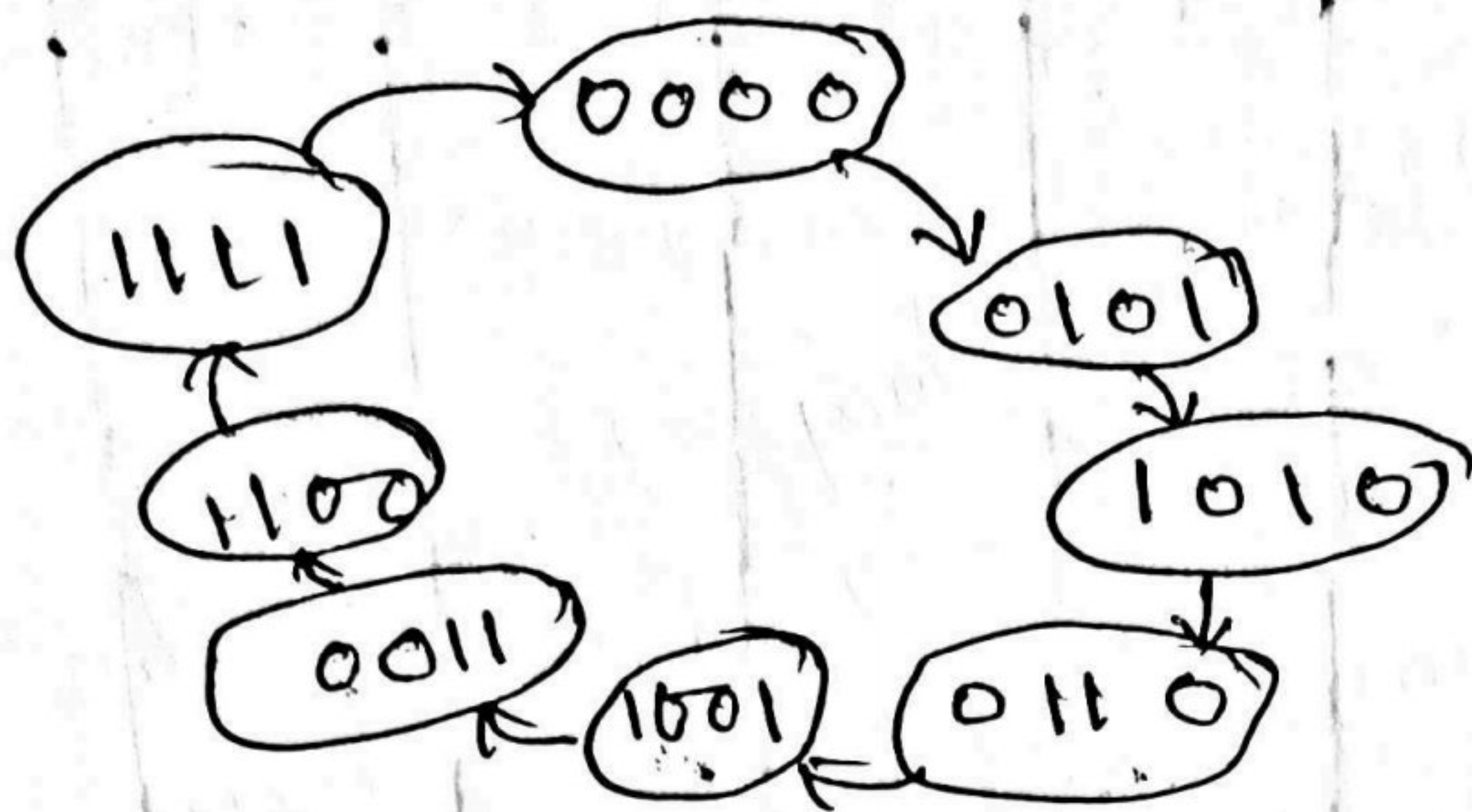


Fig-2: state diagram

Q ₁	Q ₂	Q ₃	Q ₄	After clock pulse
0	0	0	0	0
0	1	0	1	1
1	0	1	0	2
0	1	1	0	3
1	0	0	1	4
0	0	1	1	5
1	1	0	0	6
1	0	1	1	7
				8

Fig-3: Sequence table

incomplete

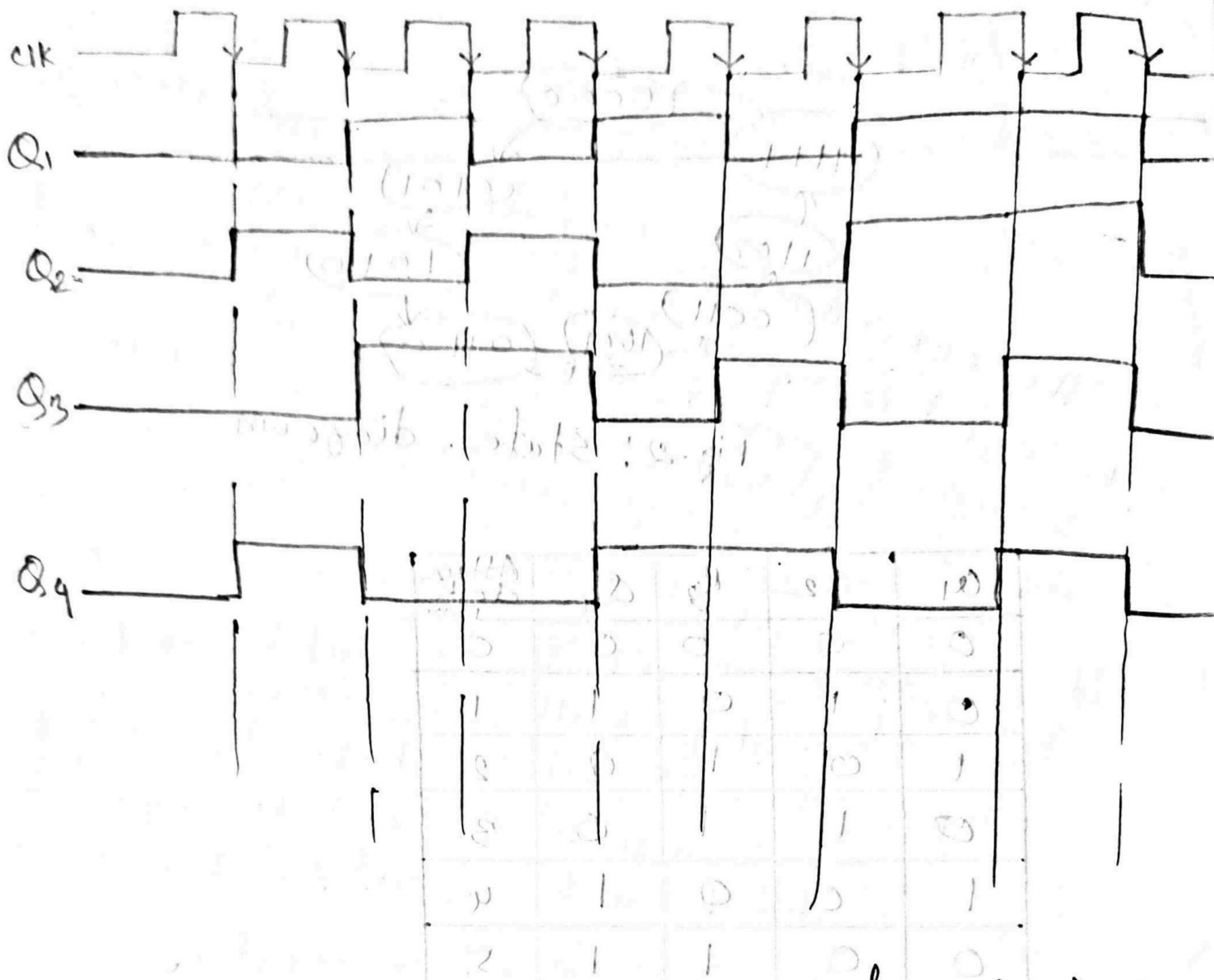


Fig: 4: Timing diagram of 4-bit twisted asynchronous counter.

Assignment no-04

Dept: CSE

Sub: Digital System Design

Roll: 2037820121

Date: 30-07-2022

17(2(c)) बूनाम मिता
बूनाम भोक् मिता?

BCD to 7 segment decoder circuit
complete वषाते 201

Dil Anot
11.09.22

2018-3(a)

Define decoder and demultiplexer. How decoder can be used as a demultiplexer?

Ans:

Decoder: A decoder is a logic circuit that accepts a set of inputs that represents a binary number and activates only the output that corresponds to that input number. All others remain inactive.

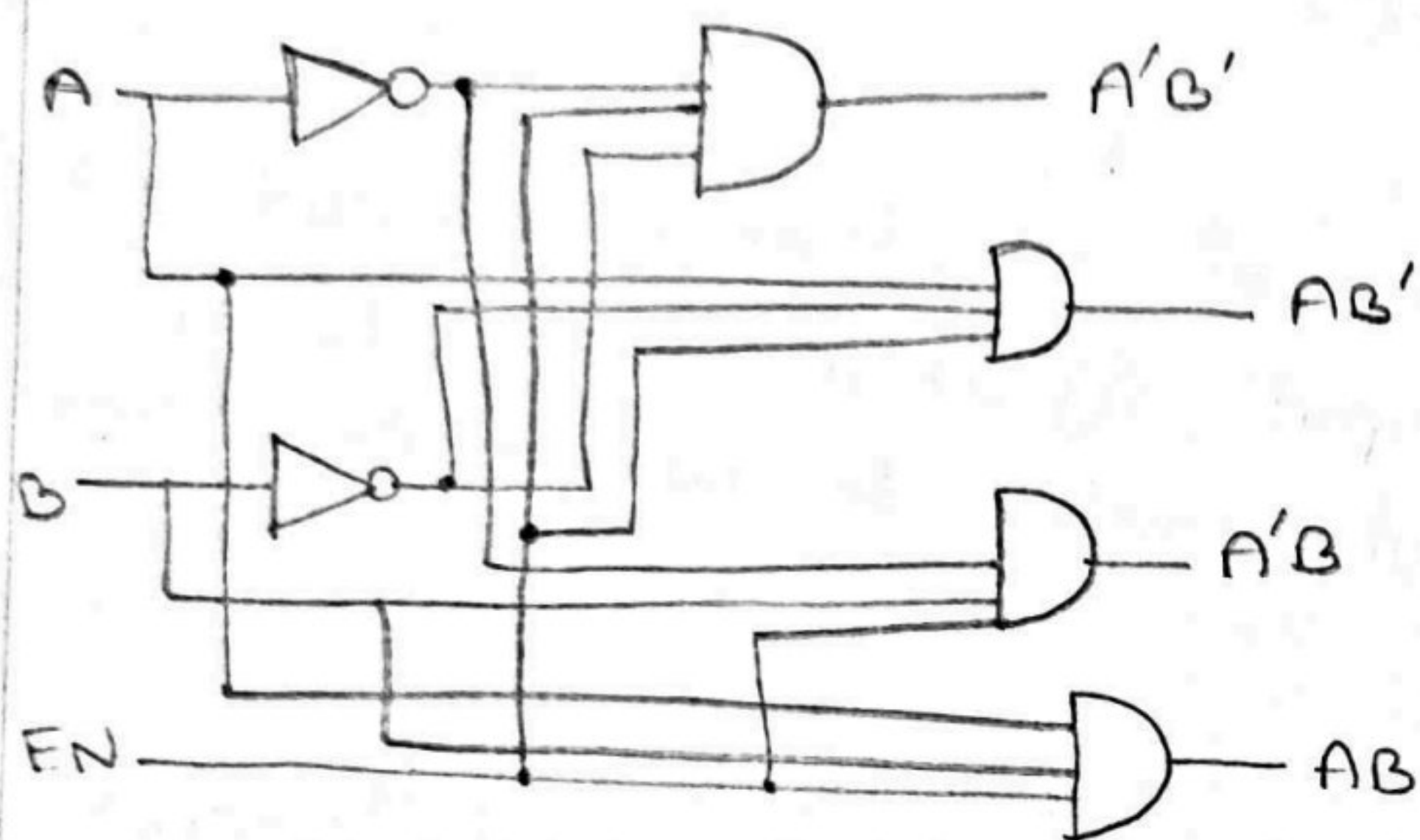
Demultiplexer:- A type of electronic circuit which receives a single input signal and selects one of multiple possible output routes to which to transmit the signal.

Decoder can be used as a Demultiplexer:-

A decoder with Enable input can function as a demux, A demux is a circuit that receives information from a single line and directs it to one of 2^n possible output lines.

Making 1:4 demux using 2:4 decoder with Enable input is possible. Let A, B be the selection lines and EN be the input line for the demultiplexer.

The decoder shown below functions as a 2:4 de-mux. When EN is taken as a data input line and A and B are taken as the selection inputs. The sig single input variable E has a path to all four outputs, but the input information is directed to only one of the output lines, as specified by the binary combination of the two selection lines A and B. This can be verified from the truth table of the circuit.



E	A	B	D ₀	D ₁	D ₂	D ₃
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

2018 - 3(b)

Define Multiplexer. Implement the logic function $F(A, B, C, D) = \sum m(1, 3, 4, 7, 11, 13, 15)$ using 8x1 mux.

Ans:- Multiplexer:- A digital multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of selection lines. Normally, there are 2^n input lines and n selection lines whose bit combinations deter-

mind which input is selected.

Truth table for 8×1 multiplexer:-

D	Input				Output
	A	B	C	D	F
	E	S ₂	S ₁	S ₀	Y
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	0
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	1
14	1	1	1	0	0
15	1	1	1	1	1

	I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇
A'	0	①	2	③	④	5	6	⑦
A	8	9	10	⑪	12	⑬	14	⑮
	0							

Fig: implementation table

Hence, the boolean expression is, $F(A, B, C, D) =$

$$\sum m(1, 3, 4, 7, 11, 13, 15).$$

This is four variable function and therefore we need a multiplexer with three selection lines and eight inputs. We choose to apply variables B, C, and D to the selection lines. The implementation table is shown above figure:-

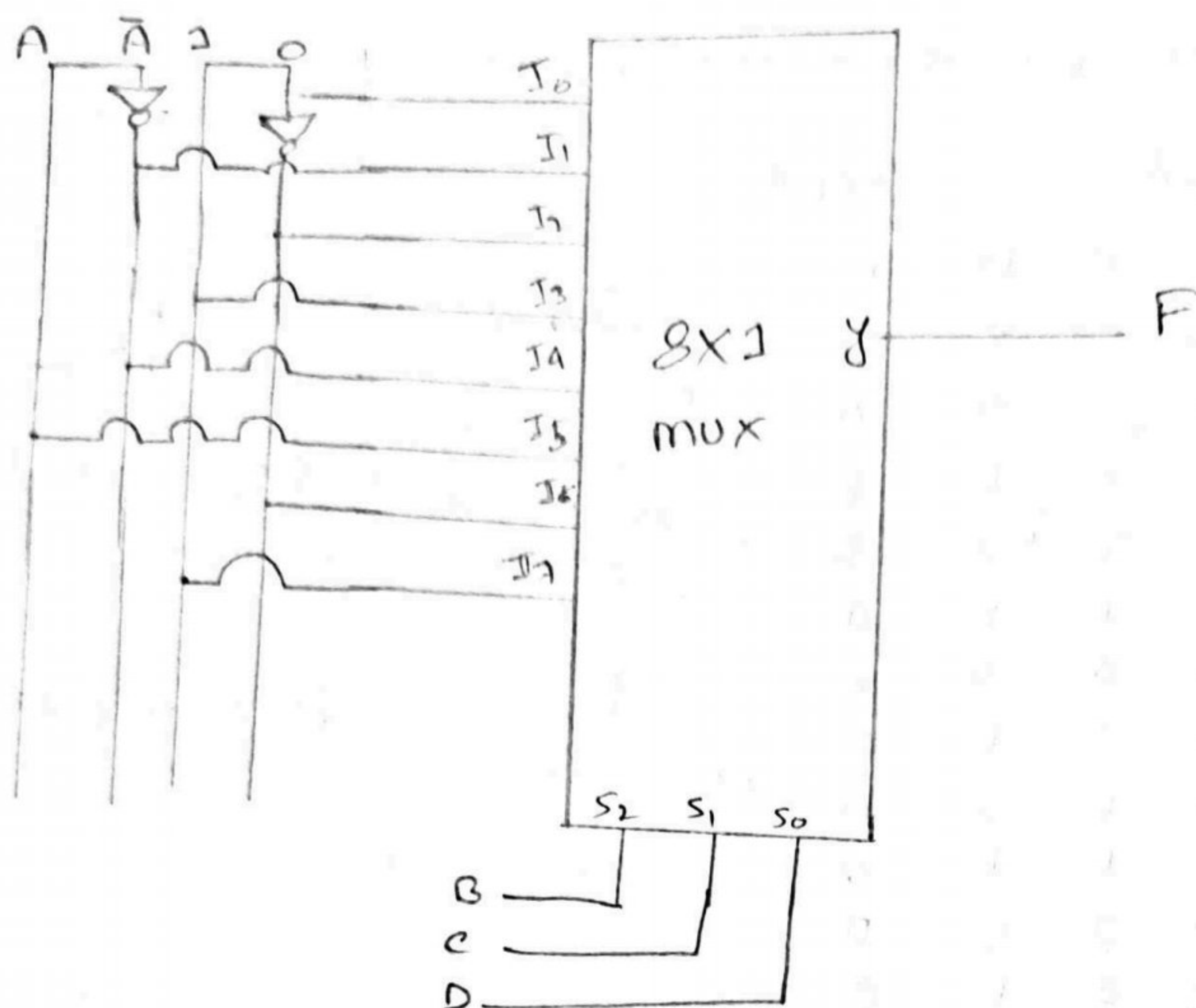


Fig: Logic circuit diagram of $F(A, B, C, D) = \sum m(1, 3, 4, 7, 11, 13, 15)$ with a 8×1 multiplexer.

2018-3(e)

Implement the logic function $F(A, B, C) = \sum m(1, 3, 4, 7)$ using a 3-to-8 line decoder with active low output.

Ans: A decoder provides the 2^n minterm of n input variables. Since any boolean function can be expressed in sum of minterms canonical form, one can use a decoder to generate the minterms and an external OR gate to form the sum. In this way,

any combinational circuit with n inputs and m outputs can be implemented with an n -to 2^n line decoder and m OR gates.

Truth table of decoder (3 to 8 line):-

Input			Output							
A	B	C	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Here, logic function $F(A, B, C) = \sum m(1, 3, 4, 7)$ using a 3-to-8 decoder with active low output.

logic circuit:-

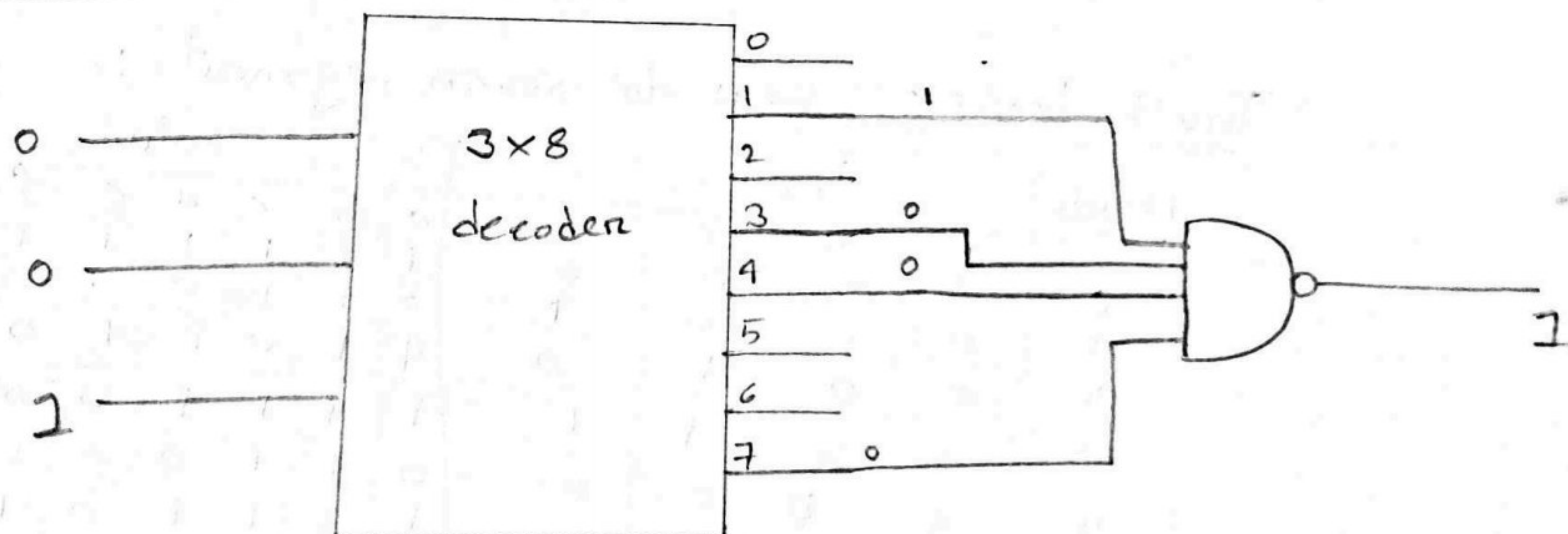
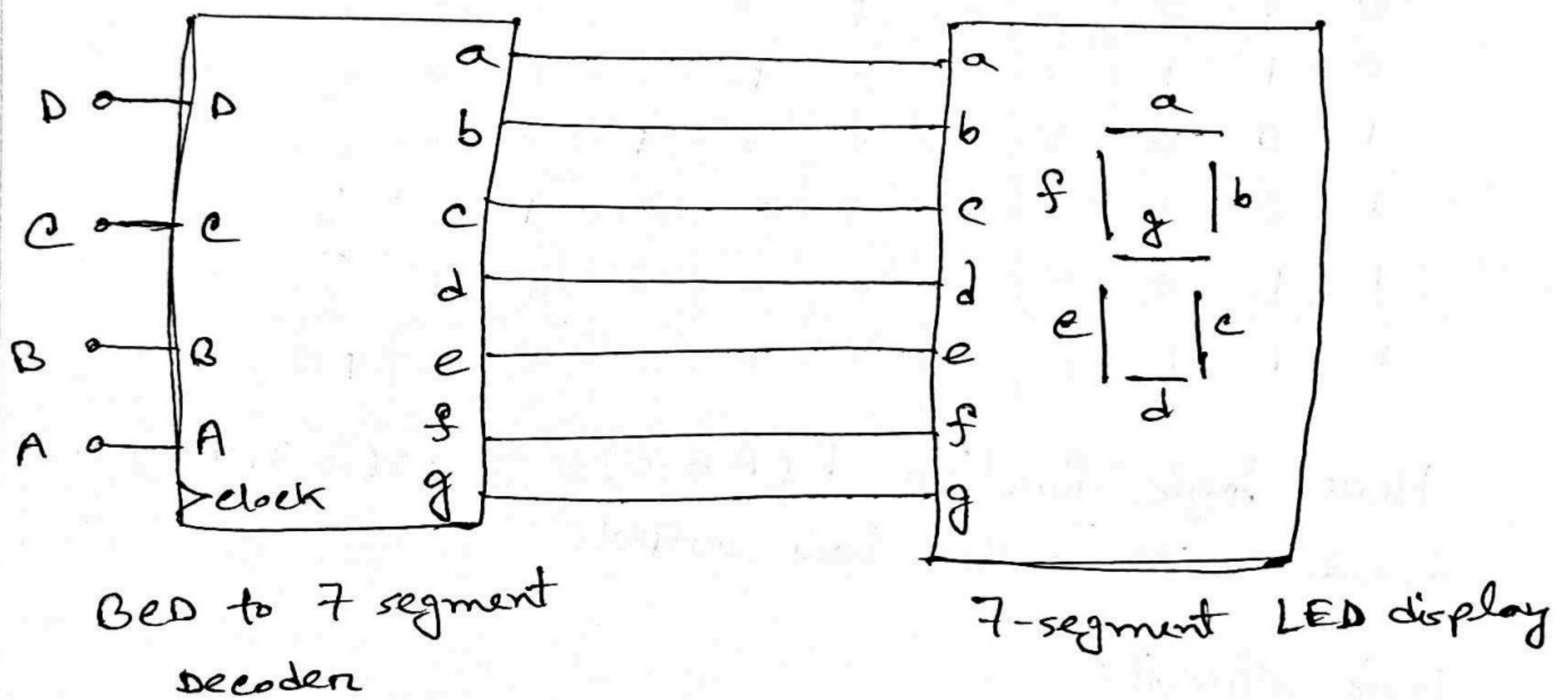


Fig: logic circuit diagram of $F(A, B, C) = \sum m(1, 3, 4, 7)$ using a active low output.

2017-2(b)

Design a combinational circuit to convert BCD code to segment code.

⇒ BCD to seven segment decoder has four input lines (A, B, C and D) and 7 output lines (a, b, c, d, e, f and g) this output is given to seven segment LED display which displays the decimal number depending upon inputs.



Truth table: BCD to seven segment :-

Digits	input				output						
	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	1	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

K-map

for a:-

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	x	x	x	x
10	1	1	x	x

$$a = A + B + BD + \bar{B}\bar{D}$$

for b:-

AB \ CD	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	x	x	x	x
10	1	1	x	x

$$b = \bar{B} + \bar{C}\bar{D} + CD$$

for c:-

AB \ CD	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	x	x	x	x
10	1	1	x	x

$$c = B + \bar{C} + D$$

for d:-

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	x	x	x	x
10	1	1	x	x

$$d = A + \bar{B}\bar{D} + \bar{B}C + C\bar{D} + B\bar{C}D$$

for e:-

AB \ CD	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	x	x	x	x
10	1	0	x	x

$$e = \bar{B}\bar{D} + C\bar{D}$$

for f:-

AB \ CD	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11	x	x	x	x
10	1	1	x	x

$$f = A + B\bar{C} + B\bar{D} + \bar{C}\bar{D}$$

Son-8:

AB \ CD	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	x	x	x	x
10	1	1	x	x

$$g = A + B\bar{C} + \bar{B}C + C\bar{D}$$

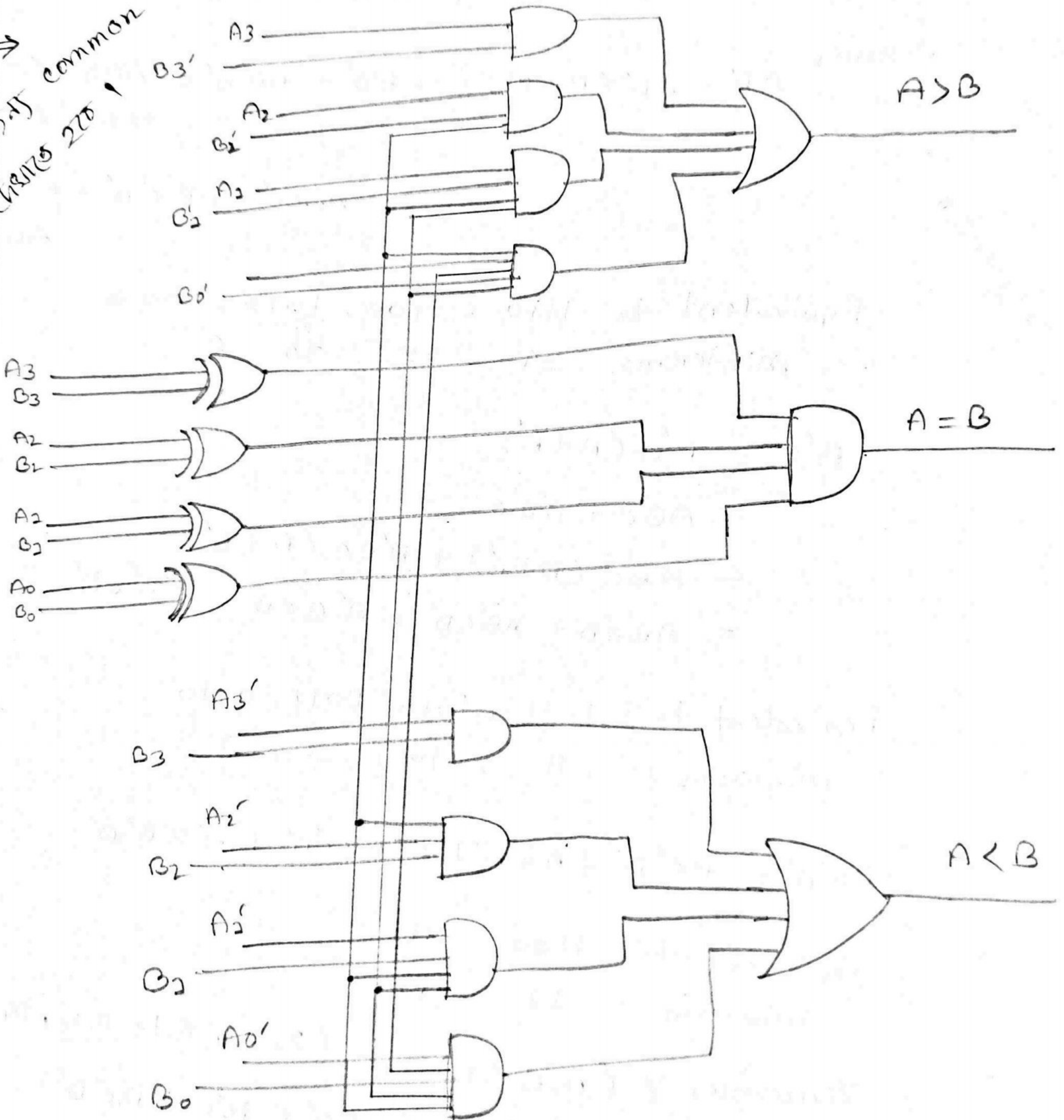
7-segment Display Decoder circuit:-



2017-2(a)

Design a combinational logic circuit to compare two 4-bit binary numbers A and B and to generate the outputs $A < B$, $A = B$ and $A > B$.

→
A B 4-bit common
line 2017-2(a)



2017-2(c)

Design a combinational logic using suitable multiplexer to realize the following boolean expression: $Y = AD' + B'e + Be'D'$.

⇒ Boolean expression: $Y = AD' + B'e + Be'D'$

Here,

$$AD' = AD'(B+B') = ABD' + AB'D' = ABD'(e+e') + AB'D'(e+e')$$

$$= ABED' + AB'eD' + AB'De' + AB'De'$$

Equivalent to 1110, 1100, 1010, 1000
Minterms 14, 12, 10, 8

$$\begin{aligned} B'e &= B'e(A+A') \\ &= AB'e + A'B'e \\ &= AB'e(D+D') + A'B'e(D+D') \\ &= AB'eD + AB'eD' + A'B'eD + A'B'eD' \end{aligned}$$

Equivalent to: 1011, 1010, 0011, 0010
Minterms: 11, 10, 3, 2

$$Be'D' = Be'D'(A+A') = AB'eD' + A'B'eD'$$

Equivalent to 1100, 0100
Minterms 12, 4

$$\begin{aligned} \text{Therefore, } Y(A, B, e, D) &= \sum (2, 3, 4, 8, 10, 11, 12, 14) \\ &= AD' + B'e + Be'D' \end{aligned}$$

प्रश्न का उत्तर

Using A, B, C as control inputs and D used as input lines of 8×2 multiplexer.

Print terms	A	B	C	D	Y	Multiplexer input
0	0	0	0	0	0	When $ABC = 000$ $Y = 0$
1	0	0	0	1	0	
2	0	0	1	0	1	When $ABC = 001$, $Y = 1$
3	0	0	1	1	1	
4	0	1	0	0	1	When $ABC = 010$ $Y = D$
5	0	1	0	1	0	
6	0	1	1	0	0	When $ABC = 011$ $Y = 0$
7	0	1	1	1	0	
8	1	0	0	0	1	When $ABC = 100$ $Y = D$
9	1	0	0	1	0	
10	1	0	1	0	1	When $ABC = 101$ $Y = 1$
11	1	0	1	1	1	
12	1	1	0	0	1	When $ABC = 110$ $Y = D$
13	1	1	0	1	0	
14	1	1	1	0	1	When $ABC = 111$ $Y = D$
15	1	1	1	1	0	

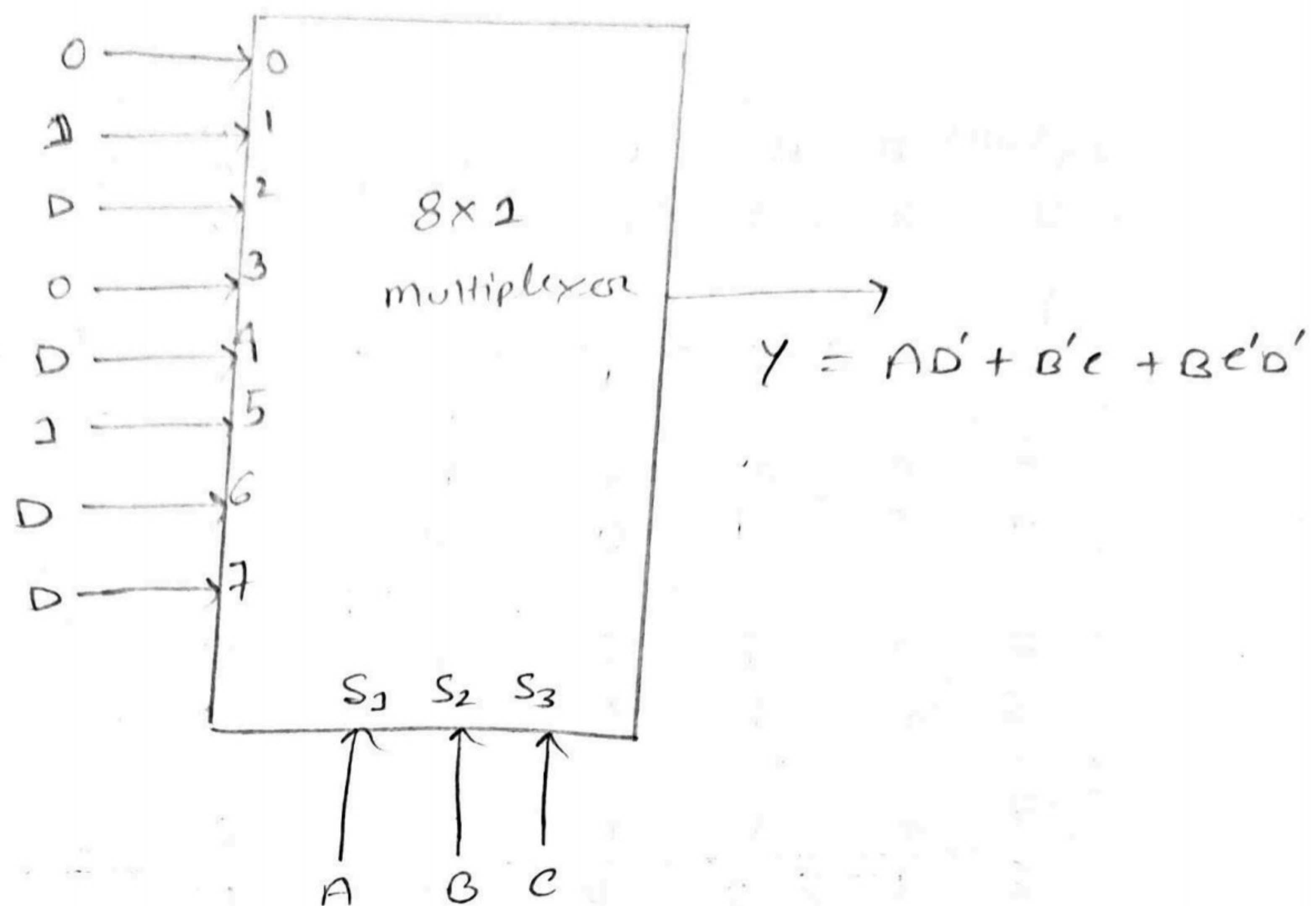


Fig: Given boolean expression using 8x2 multiplexer.

DSD

Name : Jeenat Salma

ID : 1937820106

Subject : Digital System Design

Course Code : CSE 2121

Assignment Topic : Previous year Q.
Solve

Date : 10.08.2022

2020 B(2) Jeenat

Jeenat Afroz

Ans to the Q NO 2020,3(a)

Write a HDL code for 2*2 bit combinational array multiplier.

Library ieee;

use ieee.std_logic-1164.all;

entity multiply is

port(A,B: in bit_vector (1 downto 0);

P: out bit_vector (3 downto 0)

);

end multiply;

architecture dataflow of multiply is

(Carry P₃) P₂ | P₁ P₀

begin

P(0) <= A(0) AND B(0);

P(1) <= (A(1) AND B(0)) XOR (A(0) AND B(1));

P(2) <= ((A(1) AND B(0)) AND (A(0) AND B(1))) XOR (A(1) AND B(1));

P(3) <= ((A(1) AND B(0)) AND (A(0) AND B(1))) AND (A(1) AND B(1));

end architecture.

A ₁	A ₀
B ₁	B ₀
B ₀ A ₁	B ₀ A ₀

	B ₁ A ₁	B ₁ A ₀	X
(Carry P ₃)	P ₂	P ₁	P ₀

2018 8(a)

Ans to the Q No 8(a)

Q What is VHDL? Briefly describe the salient features of VHDL & Verilog.

⇒ Very High-Speed Integrated Circuit Hardware Description language (VHDL) is a description language used to describe hardware. It is utilized in electronic design automation to express mixed-signal & digital system, ICs & FPGA (Field programmable gate array). We can also use VHDL as a general-purpose parallel programming language.

Describing the salient features of VHDL & Verilog :—

1. Concurrency: A common characteristics of both these hardware description language is that unlike software programming languages like C, java, c++ etc. Both language are concurrent in their behaviour & program execution.

Predefined construct :

VHDL & Verilog HDL has predefined operators, predefined gates, predefined resolution functions. Verilog also includes don't-care notation.

Lower level Modeling Capability :

As compared with VHDL, Verilog includes convenient truth table syntax to model primitives.

Semantics: Both VHDL & Verilog have simulation based semantics.

Compilation & Interpretation: VHDL is compiled, while Verilog is an interpretative language.

Dynamic memory allocation: VHDL supports dynamic memory allocation (pointers type), while Verilog has no features.

Roots of language: VHDL is more reliable, readable and strongly typed language with roots from ADA.

Data type: Verilog has very simple data types, while VHDL allows complex data type.

Physical Type: VHDL supports physical types, while verilog doesnot support physical type.

Conditional & Iterative generation: Both VHDL & Verilog support conditional & iterative generation.

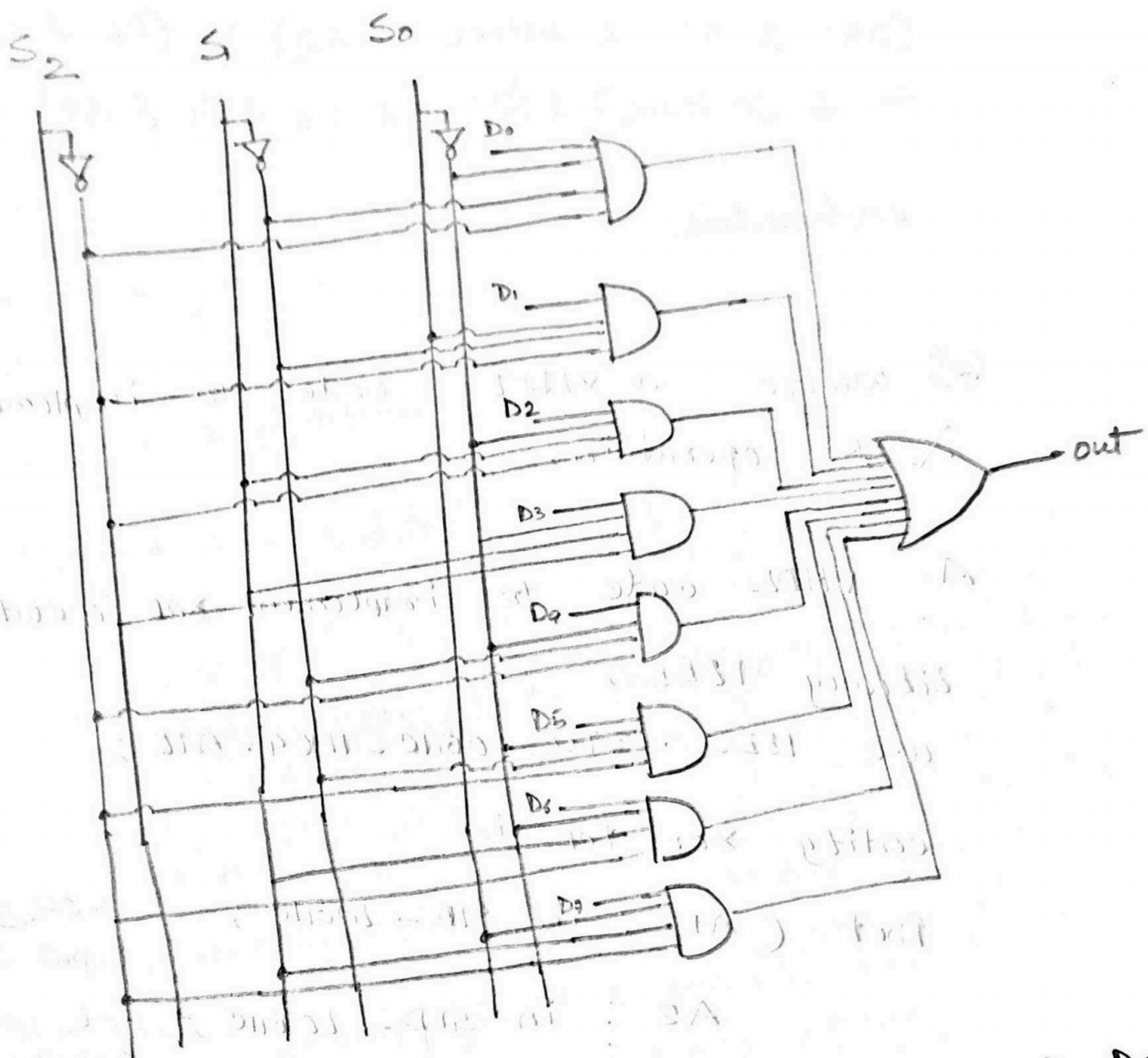
Explain 8-to-1 line multiplexer in verilog HDL.

This 8x1 mux is a high level multiplexer.

For simplicity the 8x1 mux can be implemented using 2x1 or 4x1-mux.

We observe that the input signals are D0, D1, D2, D3, D4, D5, D7, S0, S1.

and the output signal is out.



So, The code is.

```

module m_81 (output out, input D0, D1, D2,
D3, D4, D5, D6, D7, S0, S1, S2);
Assign S1_bar = ~S1;
Assign S0_bar = ~S0;
Assign S2_bar = ~S2;
Assign out = (D0 & S2_bar & S1_bar & S0_bar) +
(D1 & S2_bar & S1_bar & S0) +
(D2 & S2_bar & S1 & S0_bar) + (D3 & S2_bar & S1 & S0) +

```

$$+ (D4 \& S2 \& S1 \text{ bar} \& S0 \text{ bar}) + (D5 \& S2 \& S1 \text{ bar} \& S0) + (D6 \& S2 \& S1 \& S0 \text{ bar}) + (D7 \& S2 \& S1 \& S0)$$

end module.

② write a VHDL code to implement XOR operation.

A VHDL code to implement XOR operation.

Library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity xor-top is

Port (A1 : in STD_LOGIC; -- XOR gate input 1

A2 : in STD_LOGIC; -- XOR gate input 2);

end architecture

Md. Abu Tofail Ahamed

Roll no: 2037820101

Subject: Digital System Design

Topic: Previous Year Question Solution
(2019-2020)

Date: 02/08/2022.

Affroz
11.09.22

① @ ^{No} Yes, a 32-bit ALU as fast as a 1-bit ALU. Explanation is given below: 32-bit Arithmetic logic unit can add, subtract, multiply, divide binary numbers of 32 bits ^{at a time}. But only 1-bit binary numbers can be subtracted, multiplied, divided through. And in this 1-bit arithmetic logic unit. Case, if you work with 32-bits^{ALU} clock cycles, are much less than 1-bit^{ALU} if you want to run 32-bit operations with 1-bit, then 32-clock cycles will be generated, which takes much more time. (So, a 32-bit ALU works much faster than a 1-bit arithmetic logic unit because it takes less time.)

2019

① (b) Explain, why carry look ahead adder is faster than a regular adder.

Ans: Carry look ahead adder is faster than carry propagation adder or regular adder because —

The carry look-ahead adder calculate the carry signals in advance based on the input signals. The result is a reduced carry propagation time. [The CLA has the fastest growing area and power requirements with respect to the bit size]

✓ For a 4-bit adder, ($n=4$), the ripple carry adder or regular adder delay is $= (2n+1)T = (2 \cdot 4 + 1)T = 9T$

✓ For a 4-bit adder ($n=4$), the carry look ahead adder delay is $= nT = 4T$.

① (c) (i) Truth Table of D Flip-flop:

CLK	D	Q → Output
0	X	No Change
1	0	0
1	1	1

Let, Clock cycle is positive edge clock.

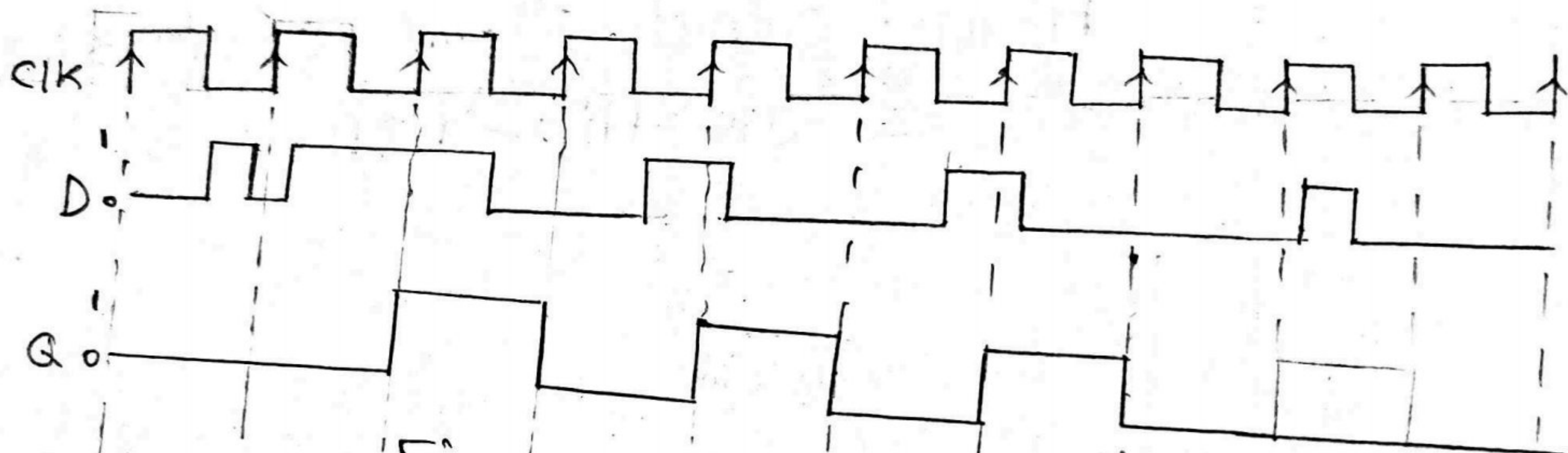


Figure: Output signal of D - Flip-flop

② Truth Table of Master-Slave JK Flip flop:

CLK	J	K	Q
0	X	X	No Change
1	0	0	No Change
1	0	1	0
1	1	0	1
1	1	1	Toggle

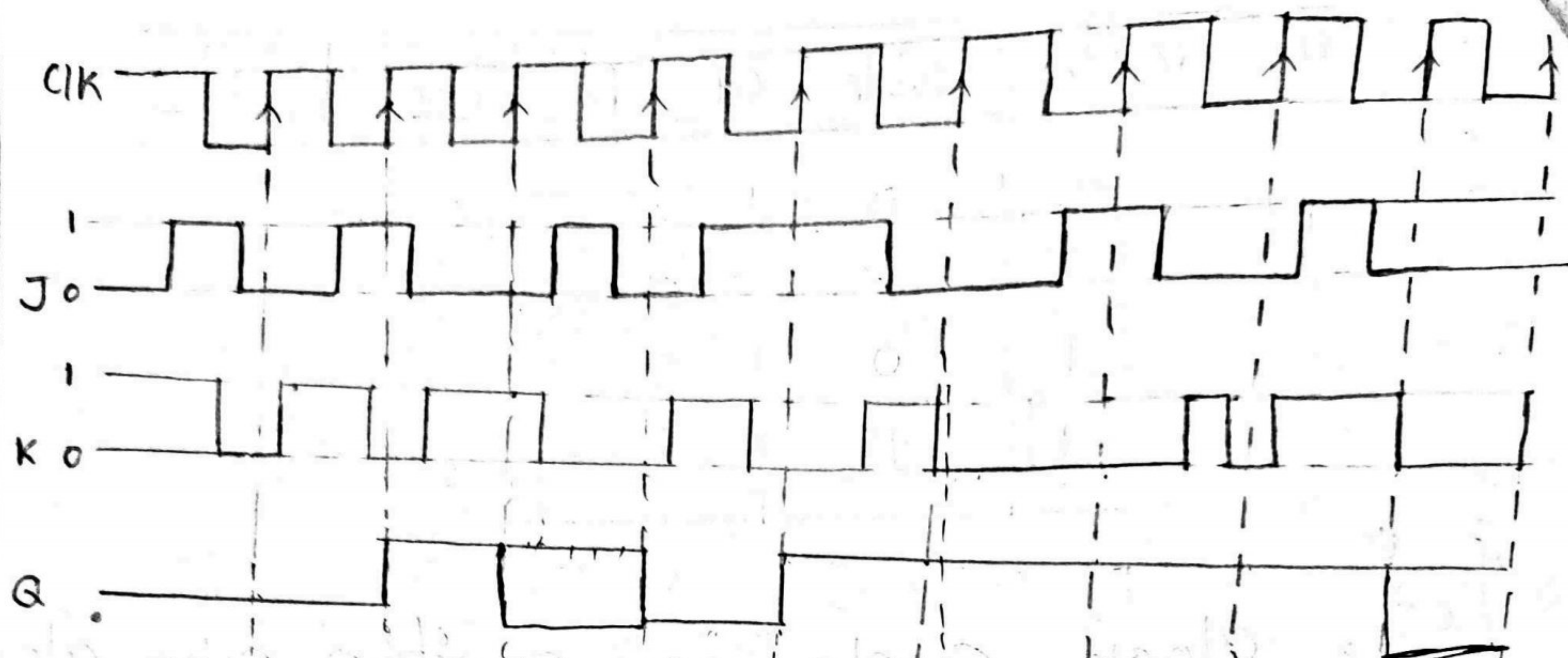


Figure: Output Signal of Master-Slave JK flip-flop.

J	K	out'Q'
0	0	no change
0	1	0
1	0	1
1	1	toggle

2019

(d) Given that,

Clk signal = 25 ms

Propagation Gate Delay = 5 ms

$J=1$, $K=1$ $\therefore J=K=1$.

Output signal, $Q = ?$

Let, Clock cycle is ~~Positive~~ edge clock.
Negative

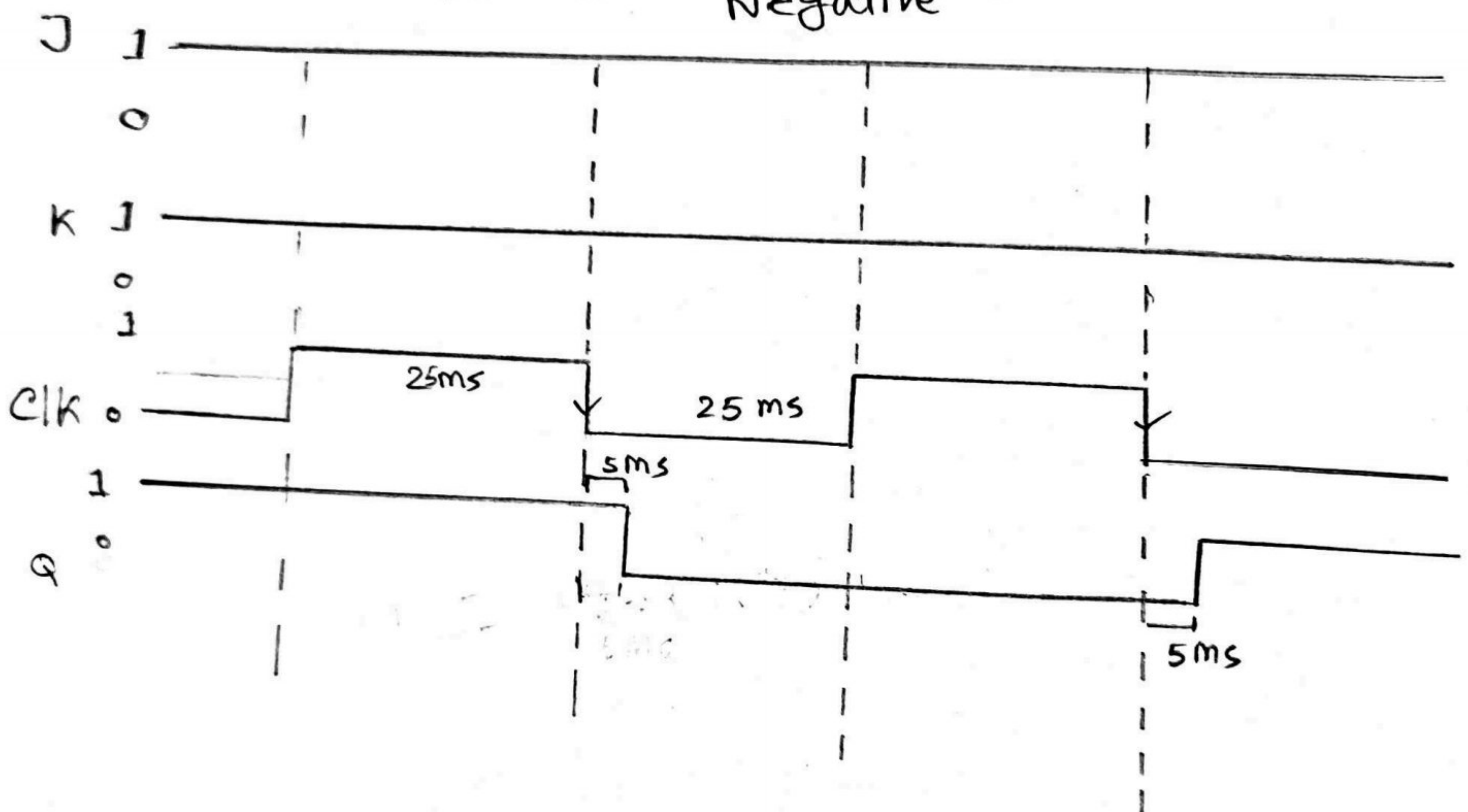


Figure: Output signal of JK flip-flop.

2020

① @ Why do we use binary number system instead of decimal number system in the digital devices?

Ans: To make sense of complicated data, your computer has to encode it in binary. Binary is a base 2 number system. On the other hand decimal is a 10 base number system. (Base 2 means there are only two digits - 1 and 0 which correspond to the on and off states your computer can understand.)

An electronic device only understand voltage and no voltage, in other words either zero or one. That's why we use base two number in the digital device.

⑥ Represent $(93)_{10}$ and $(-93)_{10}$ as 8 bit

① Sign-magnitude, ② 1's complement, and ③ 2's complement binary numbers.

Ans:

$$\begin{array}{r} 2 \overline{) 93} \\ 2 \overline{) 46} - 1 \\ 2 \overline{) 23} - 0 \\ 2 \overline{) 11} - 1 \\ 2 \overline{) 5} - 1 \\ 2 \overline{) 2} - 1 \\ 2 \overline{) 1} - 0 \\ 0 - 1 \end{array}$$

$$\therefore (93)_{10} = (1011101)_2$$

For $(-93)_{10}$:

The number $+1011101$ in 8-bit form is

01011101 .

① In 8-bit sign magnitude form it is

101011101

Sign magnitude.

② In 8-bit sign magnitude 1's Complement it is

10100010 .

③ In 8-bit sign 2's Complement it is

10100010

$+1$

10100011

(Ans.).

For $(-93)_{10}$:

(i) In 8-bit sign magnitude form it is

$\underline{11011101}$

Sign magnitude.

(ii) In 8-bit sign 1's complement it is

00100010

(iii) In 8-bit sign 2's complement it is

00100010

$+1$

00100011

① ② state the purpose of Hamming for digital communication devices.
Compute the data word with hamming code for the original binary data word $(10110010110)_2$.

Ans: Hamming code is an error connection system that can detect and correct errors when data is stored or transmitted. It requires adding additional parity bits with the data. It is commonly used in error connection code (ECC) RAM. It's named after its inventor, Richard W.

we know,

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P_1
1	0	1	1	0	0	1	-	0	1	1		0		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1

$$P_1 \rightarrow D_1 D_2 D_4 D_5 D_7 D_9 D_{11}$$

$$0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1$$

Considering Even Parity getting P_1 as below :

$$P_1 = 0$$

$$P_2 \Rightarrow D_1 D_3 D_4 D_6 D_7 D_{10} D_{11}$$

$$0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1$$

Considering Even Parity getting P_2 as below,

$$P_2 = 0.$$

$$P_4 \rightarrow D_2 D_3 D_4 D_8 D_9 D_{10} D_{11}$$

$$1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 1$$

Considering Even Parity getting P_4 as below, $P_4 = 1.$

$$P_8 = D_5 D_6 D_7 D_8 D_9 D_{10} D_{11}$$

$$= 1 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1$$

Considering Even parity getting P_8 as below:

$$P_8 = 0.$$

∴ The data word with hamming code is given below:

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P_1
1	0	1	1	0	0	1	0	0	1	1	1	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1